

Teoria de Arboles Parte 2

Recorridos en Arboles (preorden, enorden, postorden)

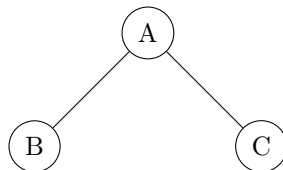
Un **recorrido** es el proceso de visitar cada nodo de un arbol exactamente una vez, en un orden particular. Los tres recorridos principales en arboles binarios son:

Recorrido en Preorden

Visita los nodos en el siguiente orden:

- Visitar la raiz.
- Recorrer el subarbol izquierdo.
- Recorrer el subarbol derecho.

Ejemplo en un arbol binario:



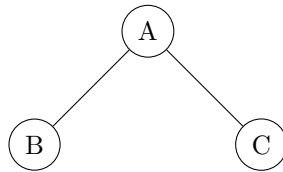
El recorrido en preorden es: A, B, C.

Recorrido en Enorden

Visita los nodos en el siguiente orden:

- Recorrer el subarbol izquierdo.
- Visitar la raiz.
- Recorrer el subarbol derecho.

Ejemplo:



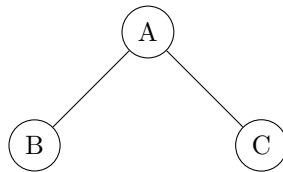
El recorrido en enorden es: B, A, C.

Recorrido en Postorden

Visita los nodos en el siguiente orden:

- Recorrer el subarbol izquierdo.
- Recorrer el subarbol derecho.
- Visitar la raiz.

Ejemplo:



El recorrido en postorden es: B, C, A.

Arboles Binarios Llenos y Completos

Arbol Binario Lleno

Un arbol binario es lleno si todos sus nodos tienen exactamente dos hijos, excepto los nodos hoja.

Arbol Binario Completo

Un arbol binario es completo si todos los niveles, excepto posiblemente el ultimo, estan completamente llenos, y todos los nodos estan lo mas a la izquierda posible.

Arboles de Expresion y su Evaluacion

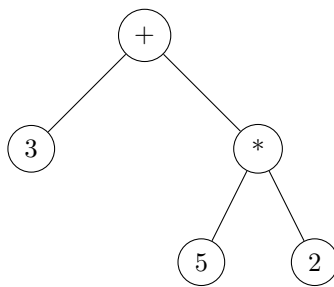
Los arboles de expresion son estructuras utilizadas para representar expresiones matematicas o logicas. Cada nodo interno representa un operador y cada nodo hoja representa un operando (constante o variable).

Evaluacion de un Arbol de Expresion

Se utiliza un recorrido postorden para evaluar un arbol de expresion:

- Recorrer el subarbol izquierdo.
- Recorrer el subarbol derecho.
- Evaluar el operador en la raiz usando los operandos resultantes de los subarboles.

Ejemplo:



El resultado de evaluar este arbol es: $3 + (5 \times 2) = 13$.

Implementacion de Arboles Binarios de Busqueda (ABB)

Los ABB son arboles binarios en los que, para cada nodo:

- Todos los valores en el subarbol izquierdo son menores que el valor del nodo.
- Todos los valores en el subarbol derecho son mayores que el valor del nodo.

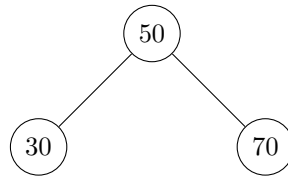
Operaciones en ABB

- **Insercion:** Se compara el valor con el nodo actual y se recorre hacia la izquierda o derecha hasta encontrar un lugar vacio.
- **Busqueda:** Similar a la insercion, pero deteniendose si se encuentra el valor buscado.
- **Eliminacion:** Tres casos posibles:
 - Nodo a eliminar es una hoja.
 - Nodo a eliminar tiene un solo hijo.
 - Nodo a eliminar tiene dos hijos: se reemplaza con el sucesor en inorden (el menor del subarbol derecho).

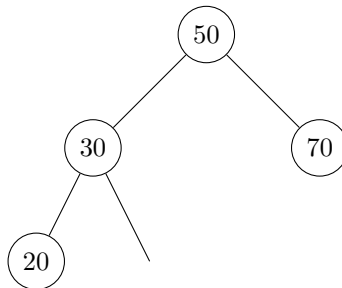
Codigo en Java para ABB:

```
1  class Nodo {
2      int valor;
3      Nodo izquierdo, derecho;
4
5      Nodo(int valor) {
6          this.valor = valor;
7          this.izquierdo = this.derecho = null;
8      }
9  }
10
11  class ArbolBinarioBusqueda {
12      Nodo raiz;
13
14      void insertar(int valor) {
15          raiz = insertarRecursivo(raiz, valor);
16      }
17
18      Nodo insertarRecursivo(Nodo nodo, int valor) {
19          if (nodo == null) {
20              return new Nodo(valor);
21          }
22
23          if (valor < nodo.valor) {
24              nodo.izquierdo = insertarRecursivo(nodo.izquierdo,
25              valor);
26          } else if (valor > nodo.valor) {
27              nodo.derecho = insertarRecursivo(nodo.derecho, valor);
28          }
29
30          return nodo;
31      }
32
33      boolean buscar(int valor) {
34          return buscarRecursivo(raiz, valor);
35      }
36
37      boolean buscarRecursivo(Nodo nodo, int valor) {
38          if (nodo == null) {
39              return false;
40          }
41          if (valor == nodo.valor) {
42              return true;
43          }
44          return valor < nodo.valor
45              ? buscarRecursivo(nodo.izquierdo, valor)
46              : buscarRecursivo(nodo.derecho, valor);
47      }
48  }
```

Ejemplo visual de insercion:



Despues de insertar 20:

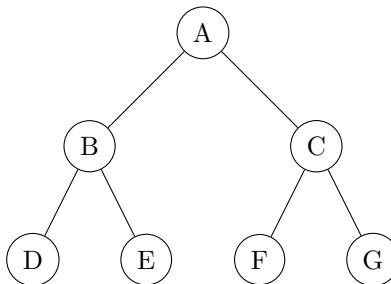


Ejercicios para Arboles Parte 2

Ejercicios Faciles

Ejercicio 1: Recorrido en Preorden

Dado el siguiente arbol binario, realiza el recorrido en preorden:



Solucion: En el recorrido en preorden, visitamos primero la raiz, luego el subarbol izquierdo, y finalmente el subarbol derecho. El resultado es:

A, B, D, E, C, F, G

Codigo en Java:

```
1 class Nodo {  
2     char valor;  
3     Nodo izquierdo, derecho;  
4  
5     Nodo(char valor) {  
6         this.valor = valor;
```

```

7         this.izquierdo = this.derecho = null;
8     }
9 }
10
11 class ArbolBinario {
12     Nodo raiz;
13
14     void preorden(Nodo nodo) {
15         if (nodo == null) {
16             return;
17         }
18         System.out.print(nodo.valor + " "); // Visitar la raiz
19         preorden(nodo.izquierdo);           // Recorrer subarbol
20         preorden(nodo.derecho);             // Recorrer subarbol
21     }
22
23     public static void main(String[] args) {
24         ArbolBinario arbol = new ArbolBinario();
25         arbol.raiz = new Nodo('A');
26         arbol.raiz.izquierdo = new Nodo('B');
27         arbol.raiz.derecho = new Nodo('C');
28         arbol.raiz.izquierdo.izquierdo = new Nodo('D');
29         arbol.raiz.izquierdo.derecho = new Nodo('E');
30         arbol.raiz.derecho.izquierdo = new Nodo('F');
31         arbol.raiz.derecho.derecho = new Nodo('G');
32
33         System.out.print("Recorrido en Preorden: ");
34         arbol.preorden(arbol.raiz);
35     }
36 }

```

Ejercicio 2: Recorrido en Enorden

Dado el mismo arbol binario, realiza el recorrido en enorden.

Solucion: En el recorrido en enorden, visitamos primero el subarbol izquierdo, luego la raiz, y finalmente el subarbol derecho. El resultado es:

D, B, E, A, F, C, G

Codigo en Java:

```

1 class ArbolBinario {
2     Nodo raiz;
3
4     void enorden(Nodo nodo) {
5         if (nodo == null) {
6             return;
7         }
8         enorden(nodo.izquierdo);           // Recorrer subarbol
9         System.out.print(nodo.valor + " "); // Visitar la raiz
10        enorden(nodo.derecho);             // Recorrer subarbol
11    }
12 }

```

```

13     public static void main(String[] args) {
14         ArbolBinario arbol = new ArbolBinario();
15         arbol.raiz = new Nodo('A');
16         arbol.raiz.izquierdo = new Nodo('B');
17         arbol.raiz.derecho = new Nodo('C');
18         arbol.raiz.izquierdo.izquierdo = new Nodo('D');
19         arbol.raiz.izquierdo.derecho = new Nodo('E');
20         arbol.raiz.derecho.izquierdo = new Nodo('F');
21         arbol.raiz.derecho.derecho = new Nodo('G');
22
23         System.out.print("Recorrido en Enorden: ");
24         arbol.enorden(arbol.raiz);
25     }
26 }

```

Ejercicio 3: Recorrido en Postorden

Dado el mismo arbol binario, realiza el recorrido en postorden.

Solucion: En el recorrido en postorden, visitamos primero el subarbol izquierdo, luego el subarbol derecho, y finalmente la raiz. El resultado es:

D, E, B, F, G, C, A

Codigo en Java:

```

1  class ArbolBinario {
2      Nodo raiz;
3
4      void postorden(Nodo nodo) {
5          if (nodo == null) {
6              return;
7          }
8          postorden(nodo.izquierdo);           // Recorrer subarbol
9                                              izquierdo
10         postorden(nodo.derecho);             // Recorrer subarbol
11                                              derecho
12         System.out.print(nodo.valor + " "); // Visitar la raiz
13     }
14
15     public static void main(String[] args) {
16         ArbolBinario arbol = new ArbolBinario();
17         arbol.raiz = new Nodo('A');
18         arbol.raiz.izquierdo = new Nodo('B');
19         arbol.raiz.derecho = new Nodo('C');
20         arbol.raiz.izquierdo.izquierdo = new Nodo('D');
21         arbol.raiz.izquierdo.derecho = new Nodo('E');
22         arbol.raiz.derecho.izquierdo = new Nodo('F');
23         arbol.raiz.derecho.derecho = new Nodo('G');
24
25         System.out.print("Recorrido en Postorden: ");
26         arbol.postorden(arbol.raiz);
27     }
28 }

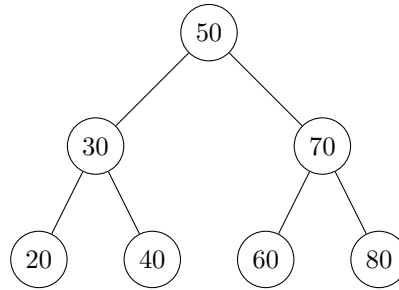
```

Ejercicios Intermedios

Ejercicio 1: Construcción de un ABB con recorrido en Preorden

Dado el conjunto de claves {50, 30, 70, 20, 40, 60, 80}, construya un Arbol Binario de Búsqueda (ABB) e imprima su recorrido en preorden.

Solucion: El ABB resultante sera:



El recorrido en preorden es: 50, 30, 20, 40, 70, 60, 80.

Código en Java:

```
1 class Nodo {
2     int valor;
3     Nodo izquierdo, derecho;
4
5     Nodo(int valor) {
6         this.valor = valor;
7         this.izquierdo = this.derecho = null;
8     }
9 }
10
11 class ArbolBinarioBusqueda {
12     Nodo raiz;
13
14     void insertar(int valor) {
15         raiz = insertarRecursivo(raiz, valor);
16     }
17
18     Nodo insertarRecursivo(Nodo nodo, int valor) {
19         if (nodo == null) {
20             return new Nodo(valor);
21         }
22         if (valor < nodo.valor) {
23             nodo.izquierdo = insertarRecursivo(nodo.izquierdo,
24                                                 valor);
25         } else if (valor > nodo.valor) {
26             nodo.derecho = insertarRecursivo(nodo.derecho, valor);
27         }
28         return nodo;
29     }
30
31     void preorden(Nodo nodo) {
32         if (nodo == null) {
33             return;
34         }
35     }
```



```

34         System.out.print(nodo.valor + " ");
35         preorden(nodo.izquierdo);
36         preorden(nodo.derecho);
37     }
38
39     public static void main(String[] args) {
40         ArbolBinarioBusqueda abb = new ArbolBinarioBusqueda();
41         int[] claves = {50, 30, 70, 20, 40, 60, 80};
42         for (int clave : claves) {
43             abb.insertar(clave);
44         }
45         System.out.print("Recorrido en Preorden: ");
46         abb.preorden(abb.raiz);
47     }
48 }

```

Ejercicio 2: Búsqueda en un ABB

Dado el ABB construido con las claves {50, 30, 70, 20, 40, 60, 80}, determine si las claves 25 y 60 se encuentran en el árbol.

Solucion:

- La clave 25 no está en el árbol.
- La clave 60 está en el árbol.

Código en Java:

```

1  class ArbolBinarioBusqueda {
2      Nodo raiz;
3
4      boolean buscar(int valor) {
5          return buscarRecursoivo(raiz, valor);
6      }
7
8      boolean buscarRecursoivo(Nodo nodo, int valor) {
9          if (nodo == null) {
10             return false;
11         }
12         if (valor == nodo.valor) {
13             return true;
14         }
15         return valor < nodo.valor
16             ? buscarRecursoivo(nodo.izquierdo, valor)
17             : buscarRecursoivo(nodo.derecho, valor);
18     }
19
20     public static void main(String[] args) {
21         ArbolBinarioBusqueda abb = new ArbolBinarioBusqueda();
22         int[] claves = {50, 30, 70, 20, 40, 60, 80};
23         for (int clave : claves) {
24             abb.insertar(clave);
25         }
26         System.out.println(" Clave 25 está en el árbol? " + abb.
27             buscar(25));
28         System.out.println(" Clave 60 está en el árbol? " + abb.
29             buscar(60));
30     }
31 }

```

29 }

Ejercicio 3: Recorrido en Enorden

Dado el ABB construido con las claves {50, 30, 70, 20, 40, 60, 80}, imprima su recorrido en enorden.

Solucion: El recorrido en enorden es: 20, 30, 40, 50, 60, 70, 80.

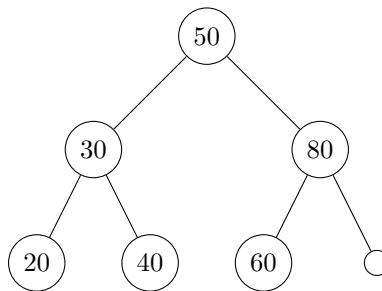
Codigo en Java:

```
1 class ArbolBinarioBusqueda {
2     void enorden(Nodo nodo) {
3         if (nodo == null) {
4             return;
5         }
6         enorden(nodo.izquierdo);
7         System.out.print(nodo.valor + " ");
8         enorden(nodo.derecho);
9     }
10
11     public static void main(String[] args) {
12         ArbolBinarioBusqueda abb = new ArbolBinarioBusqueda();
13         int[] claves = {50, 30, 70, 20, 40, 60, 80};
14         for (int clave : claves) {
15             abb.insertar(clave);
16         }
17         System.out.print("Recorrido en Enorden: ");
18         abb.enorden(abb.raiz);
19     }
20 }
```

Ejercicio 4: Eliminacion en un ABB

Dado el ABB construido con las claves {50, 30, 70, 20, 40, 60, 80}, elimine la clave 70 y muestre el arbol resultante.

Solucion: Al eliminar el nodo 70, que tiene dos hijos, lo reemplazamos por su sucesor en inorden (el menor nodo del subarbol derecho, 80). El ABB resultante es:



Codigo en Java:

```
1 class ArbolBinarioBusqueda {
2
3     Nodo eliminar(Nodo nodo, int valor) {
4         if (nodo == null) {
5             return nodo;
```

```

6         }
7
8         if (valor < nodo.valor) {
9             nodo.izquierdo = eliminar(nodo.izquierdo, valor);
10        } else if (valor > nodo.valor) {
11            nodo.derecho = eliminar(nodo.derecho, valor);
12        } else {
13            if (nodo.izquierdo == null) {
14                return nodo.derecho;
15            } else if (nodo.derecho == null) {
16                return nodo.izquierdo;
17            }
18
19            nodo.valor = minValor(nodo.derecho);
20            nodo.derecho = eliminar(nodo.derecho, nodo.valor);
21        }
22
23        return nodo;
24    }
25
26    int minValor(Nodo nodo) {
27        int minv = nodo.valor;
28        while (nodo.izquierdo != null) {
29            minv = nodo.izquierdo.valor;
30            nodo = nodo.izquierdo;
31        }
32        return minv;
33    }
34
35    public static void main(String[] args) {
36        ArbolBinarioBusqueda abb = new ArbolBinarioBusqueda();
37        int[] claves = {50, 30, 70, 20, 40, 60, 80};
38        for (int clave : claves) {
39            abb.insertar(clave);
40        }
41
42        System.out.println("Arbol despues de eliminar 70:");
43        abb.raiz = abb.eliminar(abb.raiz, 70);
44        abb.enorden(abb.raiz);
45    }
46 }

```

Ejercicio 5: Altura de un ABB

Dado el ABB construido con las claves {50, 30, 70, 20, 40, 60, 80}, calcule su altura.

Solucion: La altura del ABB es 3.

Codigo en Java:

```

1 class ArbolBinarioBusqueda {
2
3     int altura(Nodo nodo) {
4         if (nodo == null) {
5             return 0;
6         }
7         int alturaIzquierda = altura(nodo.izquierdo);
8         int alturaDerecha = altura(nodo.derecho);
9         return Math.max(alturaIzquierda, alturaDerecha) + 1;

```

```

10     }
11
12     public static void main(String[] args) {
13         ArbolBinarioBusqueda abb = new ArbolBinarioBusqueda();
14         int[] claves = {50, 30, 70, 20, 40, 60, 80};
15         for (int clave : claves) {
16             abb.insertar(clave);
17         }
18
19         System.out.println("Altura del arbol: " + abb.altura(abb.
20                             raiz));
21     }

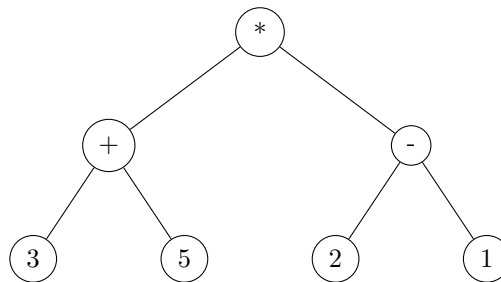
```

Ejercicios Dificiles

Ejercicio 6: Construcción de un árbol de expresión

Construya un árbol de expresión para la fórmula $(3 + 5) \times (2 - 1)$ y evalúelo.

Solucion: El árbol de expresión es:



El resultado de evaluar este árbol es $(3 + 5) \times (2 - 1) = 8 \times 1 = 8$.

Código en Java:

```

1  class Nodo {
2      String valor;
3      Nodo izquierdo, derecho;
4
5      Nodo(String valor) {
6          this.valor = valor;
7          this.izquierdo = this.derecho = null;
8      }
9  }
10
11  class ArbolDeExpresion {
12
13      int evaluar(Nodo nodo) {
14          if (nodo.izquierdo == null && nodo.derecho == null) {
15              return Integer.parseInt(nodo.valor);
16          }
17
18          int izquierdo = evaluar(nodo.izquierdo);
19          int derecho = evaluar(nodo.derecho);

```

```

20
21         switch (nodo.valor) {
22             case "+":
23                 return izquierdo + derecho;
24             case "-":
25                 return izquierdo - derecho;
26             case "*":
27                 return izquierdo * derecho;
28             case "/":
29                 return izquierdo / derecho;
30             default:
31                 throw new IllegalArgumentException("Operador
32                     invalido");
33         }
34     }
35
36     public static void main(String[] args) {
37         Nodo raiz = new Nodo("*");
38         raiz.izquierdo = new Nodo("+");
39         raiz.derecho = new Nodo("-");
40         raiz.izquierdo.izquierdo = new Nodo("3");
41         raiz.izquierdo.derecho = new Nodo("5");
42         raiz.derecho.izquierdo = new Nodo("2");
43         raiz.derecho.derecho = new Nodo("1");
44
45         ArbolDeExpresion arbol = new ArbolDeExpresion();
46         System.out.println("Resultado: " + arbol.evaluar(raiz));
47     }

```

Ejercicio 7: Conversion de notacion postfija a arbol de expresion

Dada la expresion en notacion postfija $35 + 21 - *$, conviértala en un arbol de expresion.

Solucion: El arbol de expresion es el mismo que el del ejercicio anterior.

Codigo en Java:

```

1  import java.util.Stack;
2
3  class ArbolDeExpresion {
4
5      Nodo construirDesdePostfija(String[] expresion) {
6          Stack<Nodo> pila = new Stack<>();
7
8          for (String token : expresion) {
9              if (token.matches("\\d+")) {
10                 pila.push(new Nodo(token));
11             } else {
12                 Nodo nodo = new Nodo(token);
13                 nodo.derecho = pila.pop();
14                 nodo.izquierdo = pila.pop();
15                 pila.push(nodo);
16             }
17         }
18         return pila.pop();
19     }
20
21     public static void main(String[] args) {

```

```

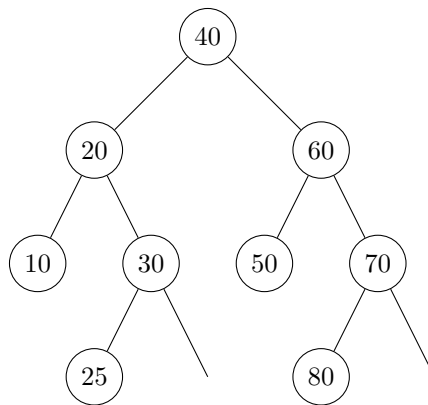
22     String[] expression = {"3", "5", "+", "2", "1", "-", "*"};
23     ArbolDeExpresion arbol = new ArbolDeExpresion();
24     Nodo raiz = arbol.construirDesdePostfija(expression);
25
26     System.out.println("Arbol construido correctamente.");
27 }
28 }

```

Ejercicio 8: Balanceo de un ABB

Dado el ABB construido con las claves {50, 30, 70, 20, 40, 60, 80, 10, 25}, balanceelo utilizando un arbol AVL.

Solucion: Despues de aplicar las rotaciones necesarias, el arbol balanceado es:



Codigo en Java:

```

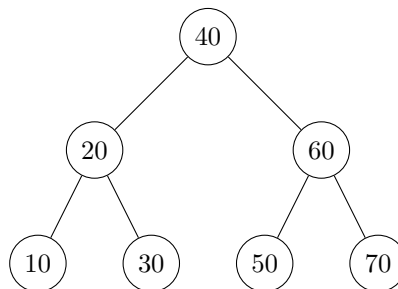
1 // Codigo completo para un arbol AVL, similar al ABB con rotaciones
  adicionales.

```

Ejercicio 9: Verificacion de un ABB

Dado un arbol binario, determine si es un Arbol Binario de Busqueda (ABB).

Arbol dado:



Solucion: El arbol cumple las propiedades de un ABB porque:

- Todos los nodos en el subarbol izquierdo de un nodo tienen valores menores.

- Todos los nodos en el subarbol derecho de un nodo tienen valores mayores.

Codigo en Java:

```

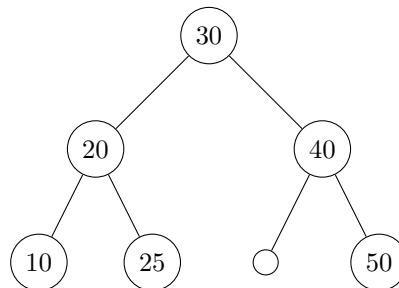
1 class ArbolBinarioBusqueda {
2
3     boolean esABB(Nodo nodo, Integer min, Integer max) {
4         if (nodo == null) {
5             return true;
6         }
7
8         if ((min != null && nodo.valor <= min) || (max != null &&
9             nodo.valor >= max)) {
10             return false;
11         }
12
13         return esABB(nodo.izquierdo, min, nodo.valor) && esABB(nodo
14             .derecho, nodo.valor, max);
15     }
16
17     public static void main(String[] args) {
18         ArbolBinarioBusqueda abb = new ArbolBinarioBusqueda();
19         Nodo raiz = new Nodo(40);
20         raiz.izquierdo = new Nodo(20);
21         raiz.derecho = new Nodo(60);
22         raiz.izquierdo.izquierdo = new Nodo(10);
23         raiz.izquierdo.derecho = new Nodo(30);
24         raiz.derecho.izquierdo = new Nodo(50);
25         raiz.derecho.derecho = new Nodo(70);
26
27         System.out.println("Es ABB: " + abb.esABB(raiz, null, null)
28             );
29     }
30 }

```

Ejercicio 10: Construcción de un arbol AVL desde un conjunto de claves

Dado el conjunto de claves {10, 20, 30, 40, 50, 25}, construya un arbol AVL y muestre su representacion final.

Solucion: El arbol AVL resultante es:



Codigo en Java:

```

1 class NodoAVL {

```

```

2      int valor, altura;
3      NodoAVL izquierdo, derecho;
4
5      NodoAVL(int valor) {
6          this.valor = valor;
7          this.altura = 1;
8      }
9  }
10
11  class ArbolAVL {
12
13      NodoAVL insertar(NodoAVL nodo, int valor) {
14          if (nodo == null) {
15              return new NodoAVL(valor);
16          }
17
18          if (valor < nodo.valor) {
19              nodo.izquierdo = insertar(nodo.izquierdo, valor);
20          } else if (valor > nodo.valor) {
21              nodo.derecho = insertar(nodo.derecho, valor);
22          } else {
23              return nodo;
24          }
25
26          nodo.altura = 1 + Math.max(altura(nodo.izquierdo), altura(
27              nodo.derecho));
28
29          int balance = obtenerBalance(nodo);
30
31          if (balance > 1 && valor < nodo.izquierdo.valor) {
32              return rotarDerecha(nodo);
33          }
34
35          if (balance < -1 && valor > nodo.derecho.valor) {
36              return rotarIzquierda(nodo);
37          }
38
39          if (balance > 1 && valor > nodo.izquierdo.valor) {
40              nodo.izquierdo = rotarIzquierda(nodo.izquierdo);
41              return rotarDerecha(nodo);
42          }
43
44          if (balance < -1 && valor < nodo.derecho.valor) {
45              nodo.derecho = rotarDerecha(nodo.derecho);
46              return rotarIzquierda(nodo);
47          }
48
49          return nodo;
50      }
51
52      int altura(NodoAVL nodo) {
53          return nodo == null ? 0 : nodo.altura;
54      }
55
56      int obtenerBalance(NodoAVL nodo) {
57          return nodo == null ? 0 : altura(nodo.izquierdo) - altura(
58              nodo.derecho);
59      }
60  }

```

```

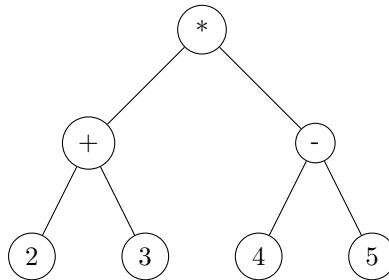
57     }
58
59     NodoAVL rotarDerecha(NodoAVL y) {
60         NodoAVL x = y.izquierdo;
61         NodoAVL T2 = x.derecho;
62
63         x.derecho = y;
64         y.izquierdo = T2;
65
66         y.altura = Math.max(altura(y.izquierdo), altura(y.derecho))
67             + 1;
68         x.altura = Math.max(altura(x.izquierdo), altura(x.derecho))
69             + 1;
70
71         return x;
72     }
73
74     NodoAVL rotarIzquierda(NodoAVL x) {
75         NodoAVL y = x.derecho;
76         NodoAVL T2 = y.izquierdo;
77
78         y.izquierdo = x;
79         x.derecho = T2;
80
81         x.altura = Math.max(altura(x.izquierdo), altura(x.derecho))
82             + 1;
83         y.altura = Math.max(altura(y.izquierdo), altura(y.derecho))
84             + 1;
85
86         return y;
87     }
88
89     public static void main(String[] args) {
90         ArbolAVL avl = new ArbolAVL();
91         NodoAVL raiz = null;
92         int[] claves = {10, 20, 30, 40, 50, 25};
93         for (int clave : claves) {
94             raiz = avl.insertar(raiz, clave);
95         }
96         System.out.println("Arbol AVL construido correctamente.");
97     }
98 }

```

Ejercicios Bonus

Ejercicio 1: Construya un arbol de expresion para la formula $(2 + 3) \times (4 - 5)$ y evaluelo.

Solucion:



Resultado: $(2 + 3) \times (4 - 5) = 5 \times -1 = -5$

Código en Java:

```
1 class Nodo {
2     String valor;
3     Nodo izquierdo, derecho;
4
5     Nodo(String valor) {
6         this.valor = valor;
7         this.izquierdo = this.derecho = null;
8     }
9 }
10
11 class ArbolDeExpresion {
12
13     int evaluar(Nodo nodo) {
14         if (nodo.izquierdo == null && nodo.derecho == null) {
15             return Integer.parseInt(nodo.valor);
16         }
17
18         int izquierdo = evaluar(nodo.izquierdo);
19         int derecho = evaluar(nodo.derecho);
20
21         switch (nodo.valor) {
22             case "+":
23                 return izquierdo + derecho;
24             case "-":
25                 return izquierdo - derecho;
26             case "*":
27                 return izquierdo * derecho;
28             case "/":
29                 return izquierdo / derecho;
30             default:
31                 throw new IllegalArgumentException("Operador
32                     invalido");
33         }
34     }
35
36     public static void main(String[] args) {
37         Nodo raiz = new Nodo("*");
38         raiz.izquierdo = new Nodo("+");
39         raiz.derecho = new Nodo("-");
40         raiz.izquierdo.izquierdo = new Nodo("2");
41         raiz.izquierdo.derecho = new Nodo("3");
42         raiz.derecho.izquierdo = new Nodo("4");
43         raiz.derecho.derecho = new Nodo("5");
```

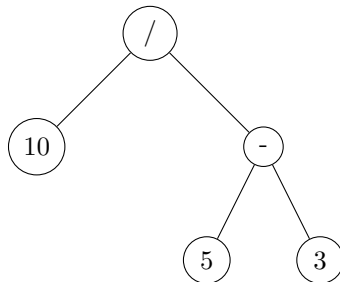
```

43     ArbolDeExpresion arbol = new ArbolDeExpresion();
44     System.out.println("Resultado: " + arbol.evaluar(raiz));
45 }
46 }
47 }

```

Ejercicio 2: Construya un arbol de expresion para la formula $10/(5 - 3)$.

Solucion:



Resultado: $10/(5 - 3) = 10/2 = 5$

Codigo en Java:

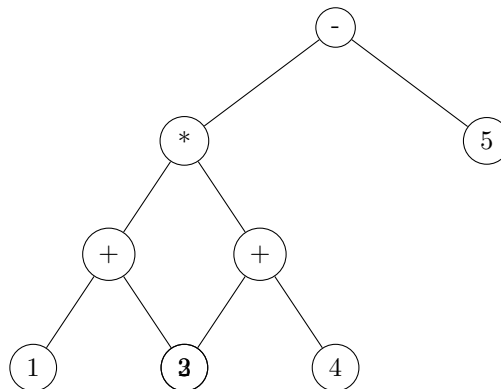
```

1 public static void main(String[] args) {
2     Nodo raiz = new Nodo("/");
3     raiz.izquierdo = new Nodo("10");
4     raiz.derecho = new Nodo("-");
5     raiz.derecho.izquierdo = new Nodo("5");
6     raiz.derecho.derecho = new Nodo("3");
7
8     ArbolDeExpresion arbol = new ArbolDeExpresion();
9     System.out.println("Resultado: " + arbol.evaluar(raiz));
10 }

```

Ejercicio 3: Construya un arbol de expresion para $((1 + 2) * (3 + 4)) - 5$.

Solucion:



Resultado: $((1 + 2) * (3 + 4)) - 5 = 21 - 5 = 16$

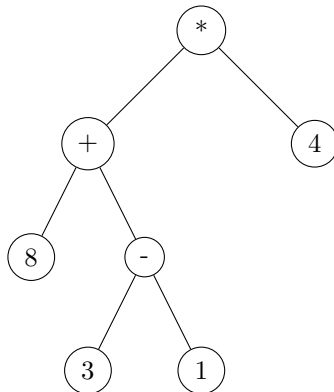
Codigo en Java:

```

1 public static void main(String[] args) {
2     Nodo raiz = new Nodo("-");
3     raiz.izquierdo = new Nodo("*");
4     raiz.derecho = new Nodo("5");
5
6     raiz.izquierdo.izquierdo = new Nodo("+");
7     raiz.izquierdo.derecho = new Nodo("+");
8
9     raiz.izquierdo.izquierdo.izquierdo = new Nodo("1");
10    raiz.izquierdo.izquierdo.derecho = new Nodo("2");
11
12    raiz.izquierdo.derecho.izquierdo = new Nodo("3");
13    raiz.izquierdo.derecho.derecho = new Nodo("4");
14
15    ArbolDeExpresion arbol = new ArbolDeExpresion();
16    System.out.println("Resultado: " + arbol.evaluar(raiz));
17 }

```

Ejercicio 4: Construya un arbol de expresion para la formula $(8+(3-1)) \times 4$.
Solucion:



Resultado: $(8 + (3 - 1)) \times 4 = 10 \times 4 = 40$

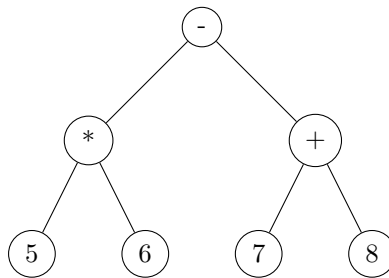
Codigo en Java:

```

1 public static void main(String[] args) {
2     Nodo raiz = new Nodo("*");
3     raiz.izquierdo = new Nodo("+");
4     raiz.derecho = new Nodo("4");
5
6     raiz.izquierdo.izquierdo = new Nodo("8");
7     raiz.izquierdo.derecho = new Nodo("-");
8     raiz.izquierdo.derecho.izquierdo = new Nodo("3");
9     raiz.izquierdo.derecho.derecho = new Nodo("1");
10
11    ArbolDeExpresion arbol = new ArbolDeExpresion();
12    System.out.println("Resultado: " + arbol.evaluar(raiz));
13 }

```

Ejercicio 5: Construya un arbol de expresion para $(5 \times 6) - (7 + 8)$.
Solucion:



Resultado: $(5 \times 6) - (7 + 8) = 30 - 15 = 15$

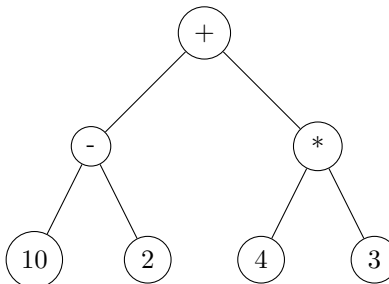
Código en Java:

```

1 public static void main(String[] args) {
2     Nodo raiz = new Nodo("-");
3     raiz.izquierdo = new Nodo("*");
4     raiz.derecho = new Nodo("+");
5
6     raiz.izquierdo.izquierdo = new Nodo("5");
7     raiz.izquierdo.derecho = new Nodo("6");
8     raiz.derecho.izquierdo = new Nodo("7");
9     raiz.derecho.derecho = new Nodo("8");
10
11     ArbolDeExpresion arbol = new ArbolDeExpresion();
12     System.out.println("Resultado: " + arbol.evaluar(raiz));
13 }
  
```

Ejercicio 6: Construya un árbol de expresión para $(10 - 2) + (4 \times 3)$.

Solución:



Resultado: $(10 - 2) + (4 \times 3) = 8 + 12 = 20$

Código en Java:

```

1 public static void main(String[] args) {
2     Nodo raiz = new Nodo("+");
3     raiz.izquierdo = new Nodo("-");
4     raiz.derecho = new Nodo("*");
5
6     raiz.izquierdo.izquierdo = new Nodo("10");
7     raiz.izquierdo.derecho = new Nodo("2");
8     raiz.derecho.izquierdo = new Nodo("4");
9     raiz.derecho.derecho = new Nodo("3");
10
  
```

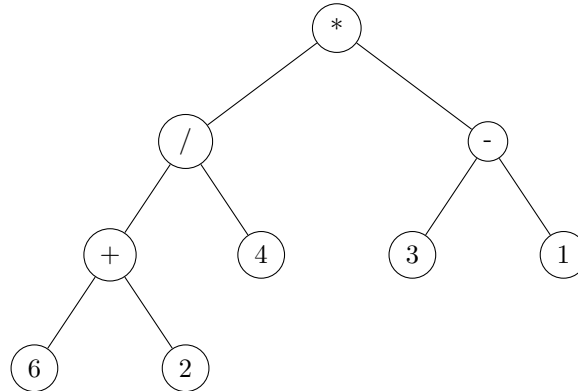
```

11     ArbolDeExpresion arbol = new ArbolDeExpresion();
12     System.out.println("Resultado: " + arbol.evaluar(raiz));
13 }

```

Ejercicio 7: Construya un arbol de expresion para la formula $((6 + 2)/4) * (3 - 1)$.

Solucion:



Resultado: $((6 + 2)/4) * (3 - 1) = (8/4) * 2 = 2 * 2 = 4$

Codigo en Java:

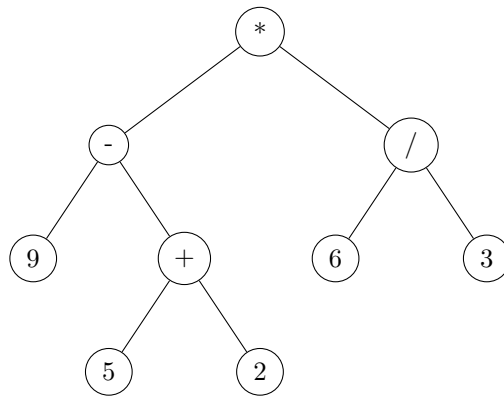
```

1 public static void main(String[] args) {
2     Nodo raiz = new Nodo("*");
3     raiz.izquierdo = new Nodo("/");
4     raiz.derecho = new Nodo("-");
5
6     raiz.izquierdo.izquierdo = new Nodo("+");
7     raiz.izquierdo.derecho = new Nodo("4");
8     raiz.izquierdo.izquierdo.izquierdo = new Nodo("6");
9     raiz.izquierdo.izquierdo.derecho = new Nodo("2");
10
11     raiz.derecho.izquierdo = new Nodo("3");
12     raiz.derecho.derecho = new Nodo("1");
13
14     ArbolDeExpresion arbol = new ArbolDeExpresion();
15     System.out.println("Resultado: " + arbol.evaluar(raiz));
16 }

```

Ejercicio 8: Construya un arbol de expresion para la formula $(9 - (5 + 2)) * (6/3)$.

Solucion:



Resultado: $(9 - (5 + 2)) * (6/3) = (9 - 7) * 2 = 2 * 2 = 4$

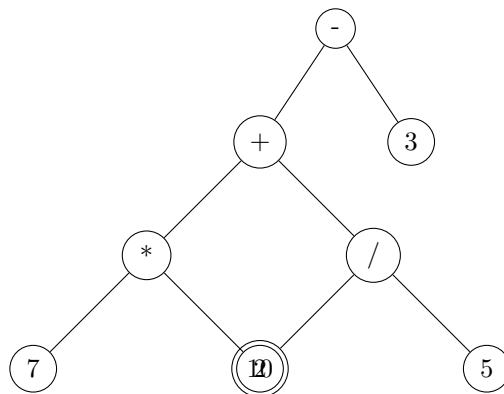
Codigo en Java:

```

1 public static void main(String[] args) {
2     Nodo raiz = new Nodo("*");
3     raiz.izquierdo = new Nodo("-");
4     raiz.derecho = new Nodo("/");
5
6     raiz.izquierdo.izquierdo = new Nodo("9");
7     raiz.izquierdo.derecho = new Nodo("+");
8     raiz.izquierdo.derecho.izquierdo = new Nodo("5");
9     raiz.izquierdo.derecho.derecho = new Nodo("2");
10
11     raiz.derecho.izquierdo = new Nodo("6");
12     raiz.derecho.derecho = new Nodo("3");
13
14     ArbolDeExpresion arbol = new ArbolDeExpresion();
15     System.out.println("Resultado: " + arbol.evaluar(raiz));
16 }
  
```

Ejercicio 9: Construya un arbol de expresion para la formula $((7 * 2) + (10/5)) - 3$.

Solucion:



Resultado: $((7 * 2) + (10/5)) - 3 = (14 + 2) - 3 = 16 - 3 = 13$

Codigo en Java:

```
1 public static void main(String[] args) {
2     Nodo raiz = new Nodo("-");
3     raiz.izquierdo = new Nodo("+");
4     raiz.derecho = new Nodo("3");
5
6     raiz.izquierdo.izquierdo = new Nodo("*");
7     raiz.izquierdo.derecho = new Nodo("/");
8
9     raiz.izquierdo.izquierdo.izquierdo = new Nodo("7");
10    raiz.izquierdo.izquierdo.derecho = new Nodo("2");
11    raiz.izquierdo.derecho.izquierdo = new Nodo("10");
12    raiz.izquierdo.derecho.derecho = new Nodo("5");
13
14    ArbolDeExpresion arbol = new ArbolDeExpresion();
15    System.out.println("Resultado: " + arbol.evaluar(raiz));
16 }
```